

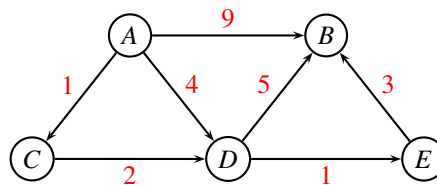
Solution, exam 18 December, 2019.

Course title: Network Optimization

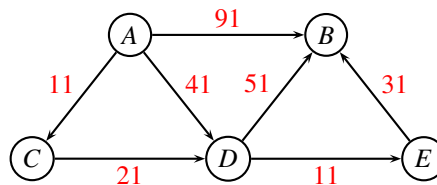
Course no: 42115

Shortest path problem

Directed graph where distances w_{ij} are marked with red.



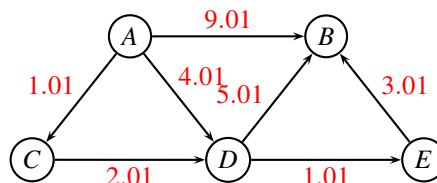
Answer 1 Choose a big number M larger than n (number of nodes). Introduce new edge weights $w'_{ij} = Mw_{ij} + 1$. In our case we can choose $M = 10$ getting:



In all calculations, the most significant digit will represent the distance, while the least significant digit will represent the number of edges. Since the graph has 5 nodes a shortest path can contain at most 4 edges, hence a carry will never propagate to the most significant digit.

Alternative formulation

Introduce new edge weights $w'_{ij} = w_{ij} + \epsilon$, where ϵ is a small number, e.g. 0.01. If two solutions have the same cost, then ϵ will count the number of edges:

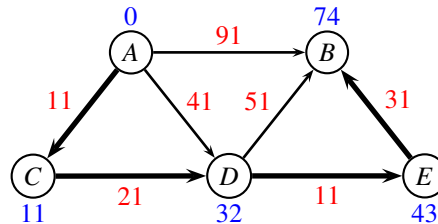


As can be seen, this is equivalent to the first transformation, but we need to use non-integer numbers in our calculations. ■

Answer 2 Since all costs are positive, we can use Dijkstra's algorithm. The following table states for each iteration which node is considered, and what the so far best distances are to each node. When a node has been considered, the distance is not changed any more.

considered node	distance d_i				
	A	B	C	D	E
init	0	∞	∞	∞	∞
A		91	11	41	
C				32	
D		83			43
E		74			
B					

The shortest path tree is marked with fat edges below (shortest distance d_i marked with blue):

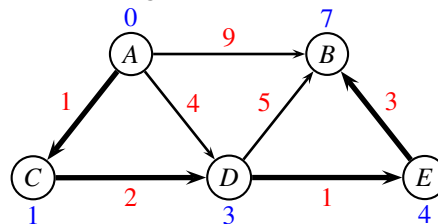


The first digit of d_i denotes distance, second digit denotes number of edges. ■

If Problem 1 was not solved, running Dijkstra's algorithm on original graph gives:

considered node	distance d_i				
	A	B	C	D	E
init	0	∞	∞	∞	∞
A		9	1	4	
C				3	
D		8			4
E		7			
B					

The shortest path tree is marked with fat edges below (shortest distance d_i marked with blue):



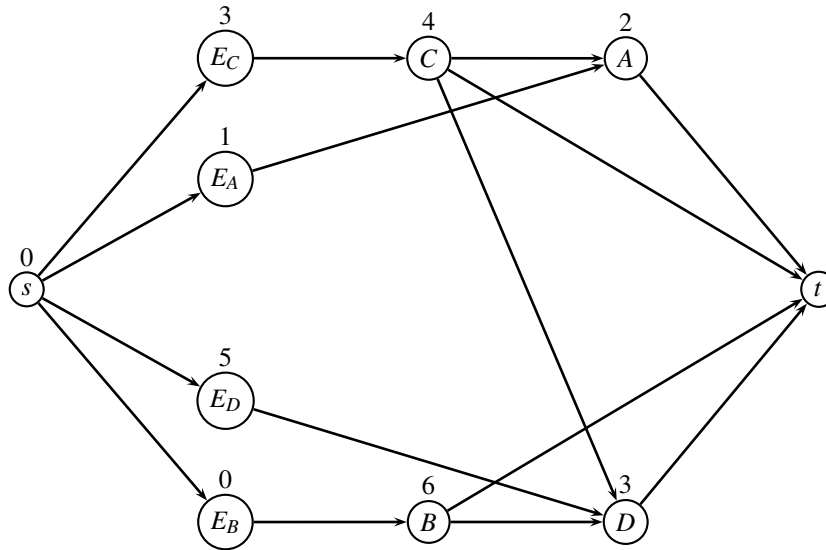
Critical path with time windows

Answer 3 For each activity i add an extra activity E_i with processing time e_i . Add E_i to the list of predecessors for activity i . All the activities E_i have s as predecessor. ■

Answer 4 With the above transformation, we get the problem:

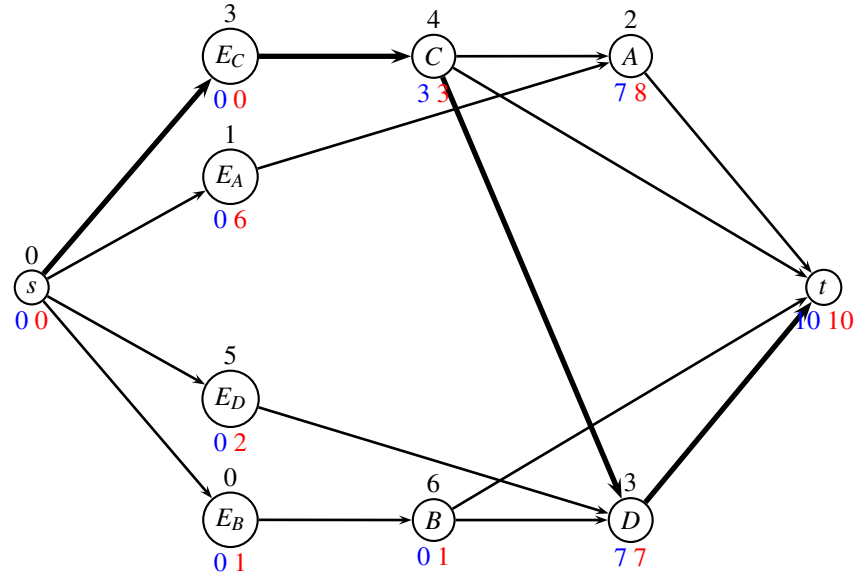
activity i	processing time p_i	predecessors
A	2	C, E_A
B	6	E_B
C	4	E_C
D	3	B, C, E_D
E_A	1	s
E_B	0	s
E_C	3	s
E_D	5	s

This leads to the topologically ordered graph, where processing time p_i is indicated on the nodes



Now we solve the problem

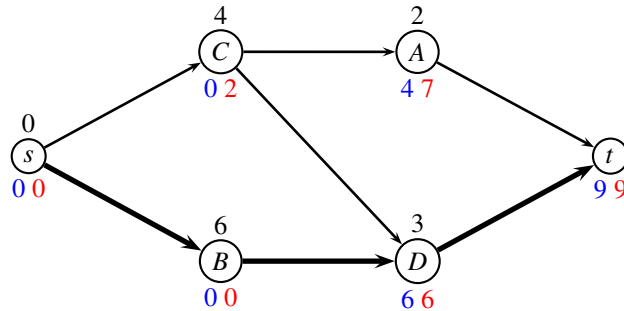
- *Forward procedure:* Consider the nodes j in topological order. Starting with $U_s = 0$, calculate the earliest time U_j each activity j can be started $U_j = \max_{(i,j) \in E} \{U_i + p_i\}$. (U_j is indicated with blue)
- *Makespan:* $T = U_t = 10$.
- *Backward procedure:* Consider the nodes j in backward topological order. Starting with $V_t = T$, calculate the latest time V_j each activity j can be started so that this makespan is realized. $V_i = \min_{(i,j) \in E} \{V_j - p_i\}$. (V_j is indicated with red)



■

Answer 5 Nodes with $U_j = V_j$ define the critical path. It is indicated with fat edges on the figure. The critical path will not necessarily be on the tasks. As seen in the example, it can also be defined by the e_j nodes. ■

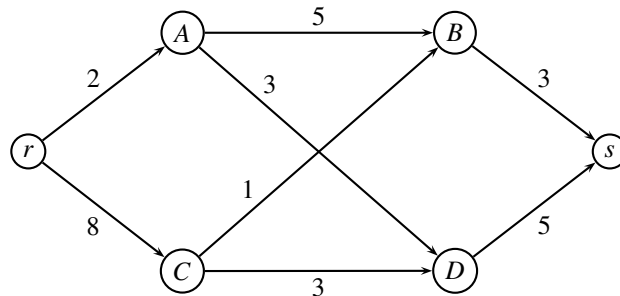
If Problem 3 was not solved, running critical path algorithm on original graph gives:



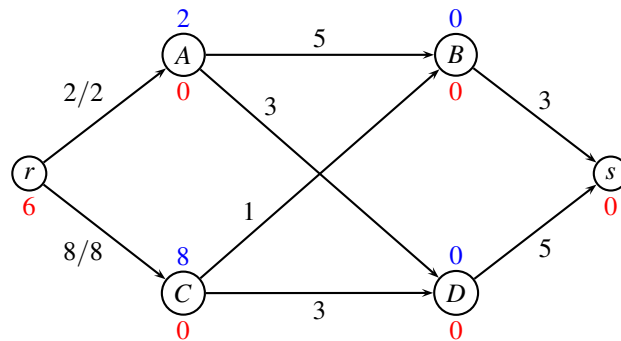
Makespan: $T = U_t = 9$. Critical path is indicated with fat edges.

Maximum flow problem

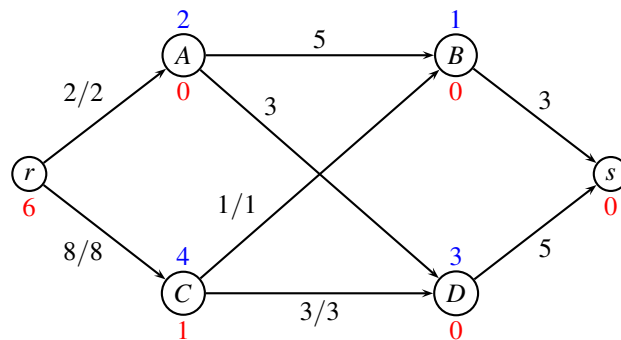
We have the maximum flow problem with capacity u_{ij} for each edge $(i, j) \in E$.



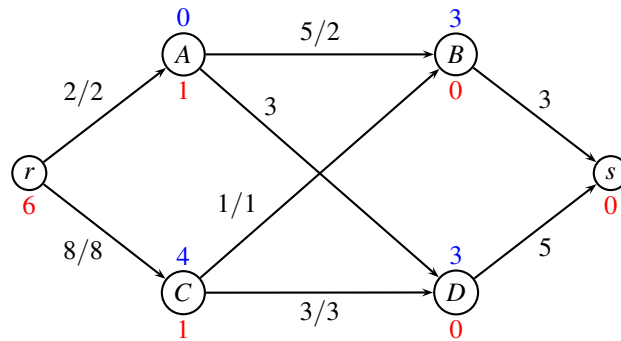
Answer 6 Initialize: For node i , red value is label d_i , blue value is flow excess $f_x(i)$



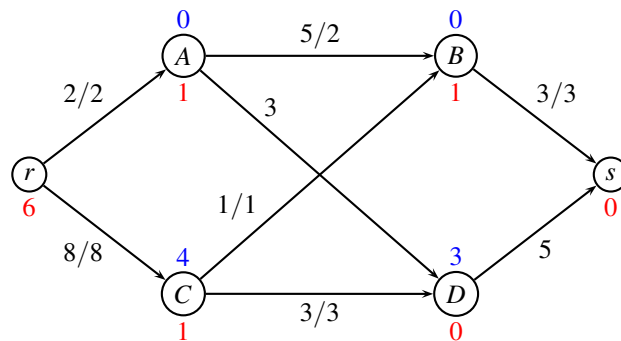
Iteration 1: relabel C, push



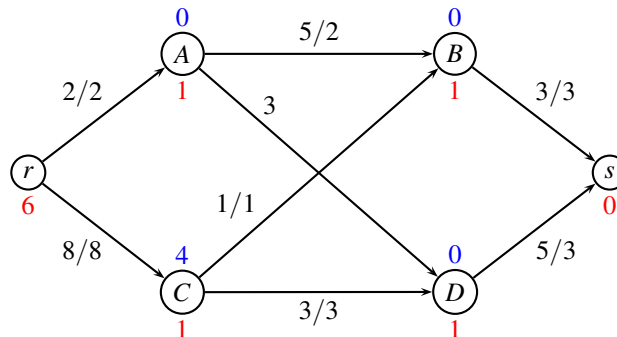
Iteration 2: relabel A, push



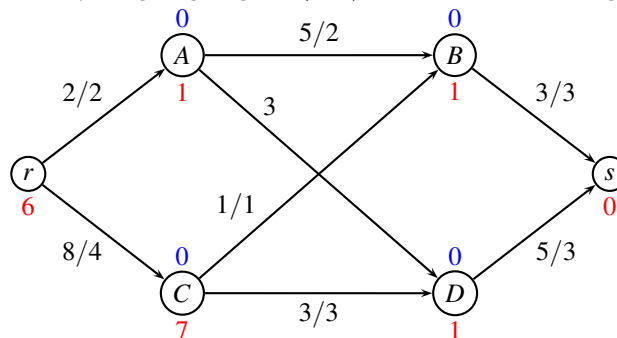
Iteration 3: relabel B, push



Iteration 4: relabel D , push



Iteration 5: relabel C (The only outgoing edge is (C, r) hence label is one bigger than label of r), push



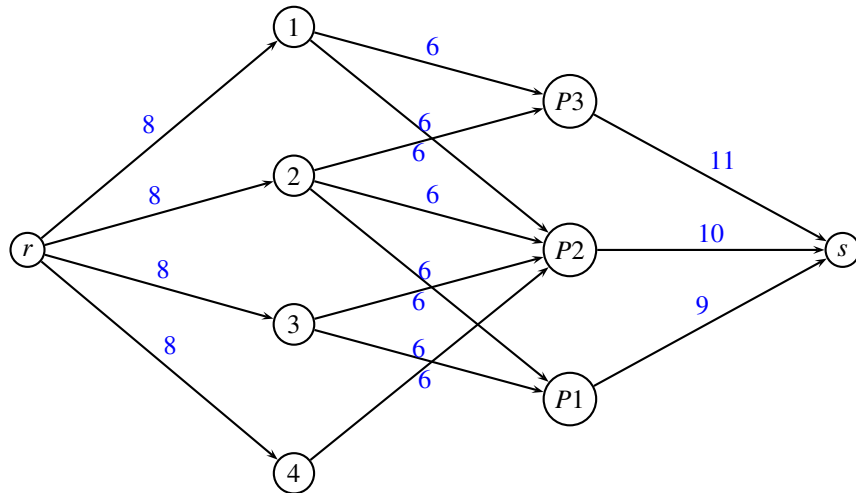
Flow is feasible, hence we stop. ■

Answer 7 A minimum cut is found by selecting an unused label value, e.g. $k = 2$. Then we have $R = \{i \mid d_i > k\} = \{r, C\}$ and $\bar{R} = \{i \mid d_i < k\} = \{A, B, D, s\}$. The capacity of the cut is $u(R) = 2 + 1 + 3 = 6$. ■

Workforce plan

Answer 8 We formulate the problem as a maximum flow problem: We introduce nodes 1, 2, 3, 4 corresponding to the four months m . Moreover we introduce nodes $P1, P2, P3$ for the projects. Each project Pi has an edge from month m if the month is within the the time window of the project. These edges have capacity 6 since only 6 engineers can work on a project in a given month. Each month m has an edge from s with capacity 8, since only 8 engineers are available each month. Finally, each project Pi has an edge to t with capacity equal to the required man-months.

The following network shows the resulting maximum flow problem:



If we can find a flow of value 30, it is possible to assign the correct assignment of engineers. The flow on edges between month m and project P_i say how many engineers should be assigned to the project in the given month. ■

Repositioning containers

Answer 9 We first calculate the demand of containers (in TEU) as

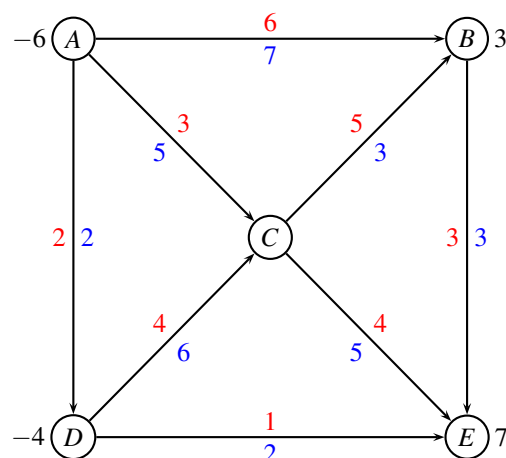
$$d_i = - \sum_{t=1,2,3} \sigma_{ti} \cdot w_i$$

where t is the container type (20", 40", high-cube) and w_i is the size of the container measured in TEU. This gives the demands

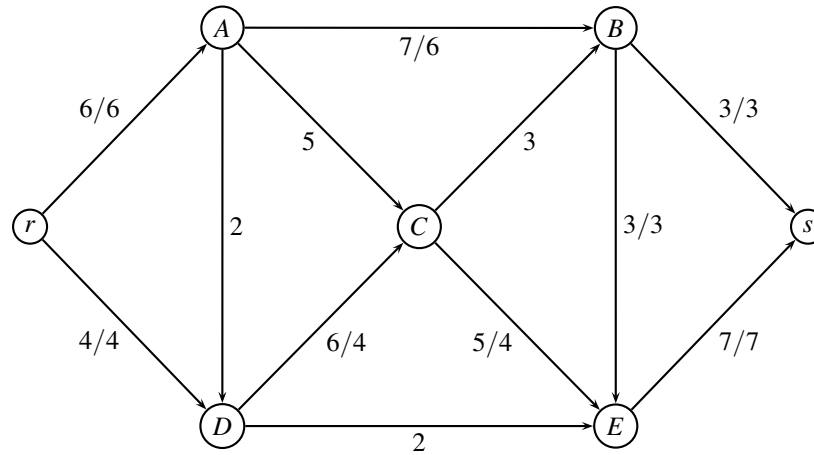
Node i	A	B	C	D	E
Demand d_i	-6	3	0	-4	7

Now, we can solving a minimum cost flow problem with the above demands. ■

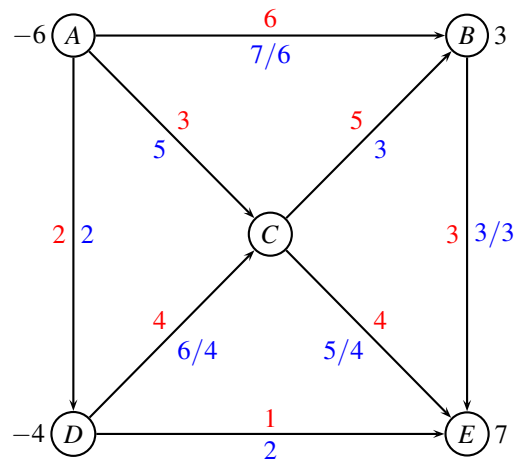
Answer 10 We have the following network



To find an initial solution, we solve a maximum flow problem. Augmenting paths: Path $rABs$ with $\delta = 3$. Path $rABEs$ with $\delta = 3$. Path $rDCEs$ with $\delta = 4$.



We now have the initial flow, and we can forget the demands:



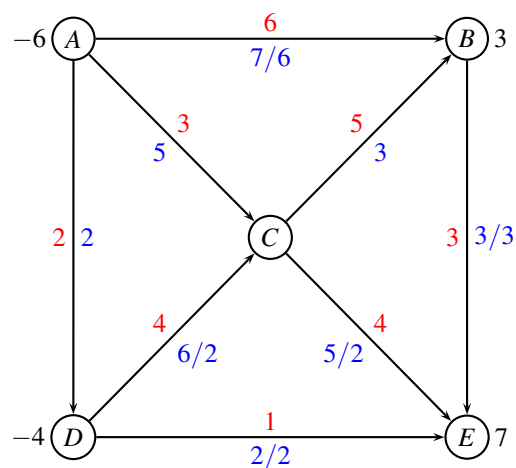
cost: $z = 6 \cdot 6 + 3 \cdot 3 + 4 \cdot 4 + 4 \cdot 4 = 77$

cycle: $C = \{DECD\}$

$w(C) = 1 - 4 - 4 = -7$

$\delta = \min\{2, 4, 4\} = 2$

Sending δ units along cycle C



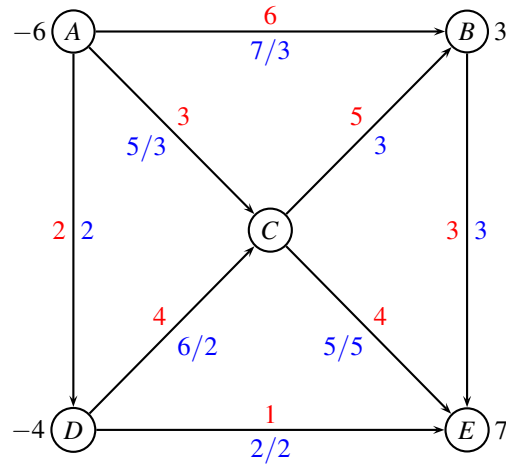
cost: $z = 6 \cdot 6 + 3 \cdot 3 + 4 \cdot 2 + 4 \cdot 2 + 2 \cdot 2 = 63$

cycle: $C = \{ACEBA\}$

$w(C) = 3 + 4 - 3 - 6 = -2$

$\delta = \min\{5, 3, 3, 6\} = 3$

Sending δ units along cycle C



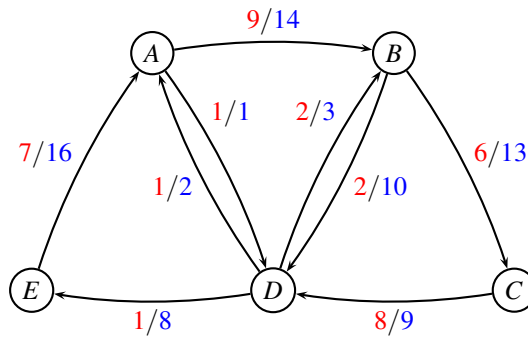
cost: $z = 6 \cdot 3 + 3 \cdot 3 + 4 \cdot 2 + 4 \cdot 5 + 2 \cdot 2 = 57$

no more cycles

stop ■

Multicommodity-flow problem

MEDITLINE (Route net of a Mediterranean liner shipping company)

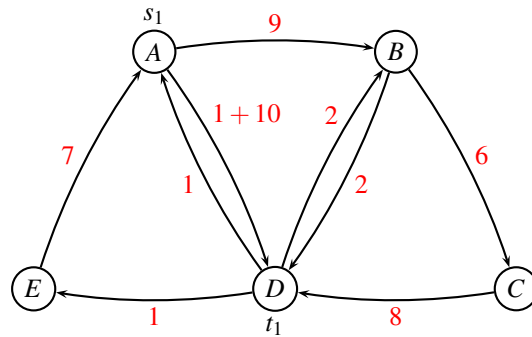


k	s_k	t_k	d_k	w_k
1	A	D	3	1
2	E	D	4	2
3	B	A	2	4

The dual variables are $y_{AD} = -10$ while $y_{ij} = 0$ for all other edges (i, j) . Moreover, $\bar{y}_1 = 11$, $\bar{y}_2 = 36$, $\bar{y}_3 = 100$.

Answer 11 For **commodity 1** we remove all edges having capacity less than $w_1 = 1$ getting the

shortest path problem:

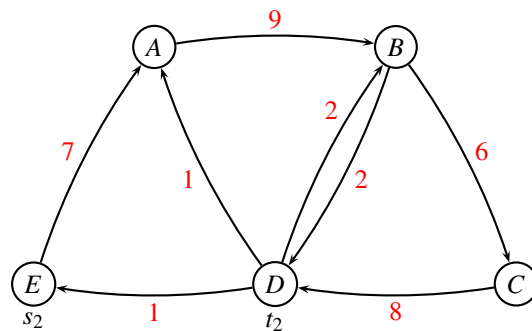


We solve the shortest path problem using Dijkstra:

considered node	distance				
	A	B	C	D	E
init	0	∞	∞	∞	∞
A		9		11	
B			15		
D					12
E					
C					

The shortest path is AD with distance $z = 11$. The reduced cost is now $\bar{c}_1 = z \cdot w_1 - \bar{y}_1 = 11 \cdot 1 - 11 = 0$.

For **commodity 2** we remove all edges having capacity less than $w_2 = 2$ getting the shortest path problem:

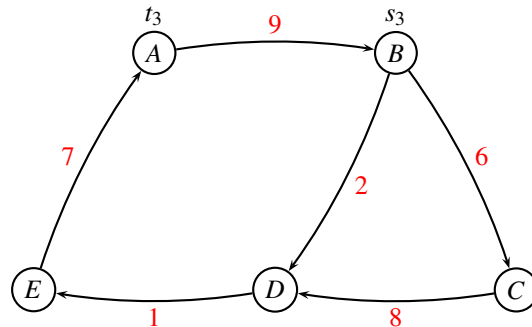


We solve the shortest path problem using Dijkstra:

considered node	distance				
	A	B	C	D	E
init	∞	∞	∞	∞	0
E	7				
A		16			
B			22	18	
D					
C					

The shortest path is $EABD$ with distance $z = 18$. The reduced cost is now $\bar{c}_2 = z \cdot w_2 - \bar{y}_2 = 18 \cdot 2 - 36 = 0$.

For **commodity 3** we remove all edges having capacity less than $w_3 = 4$ getting the shortest path problem:



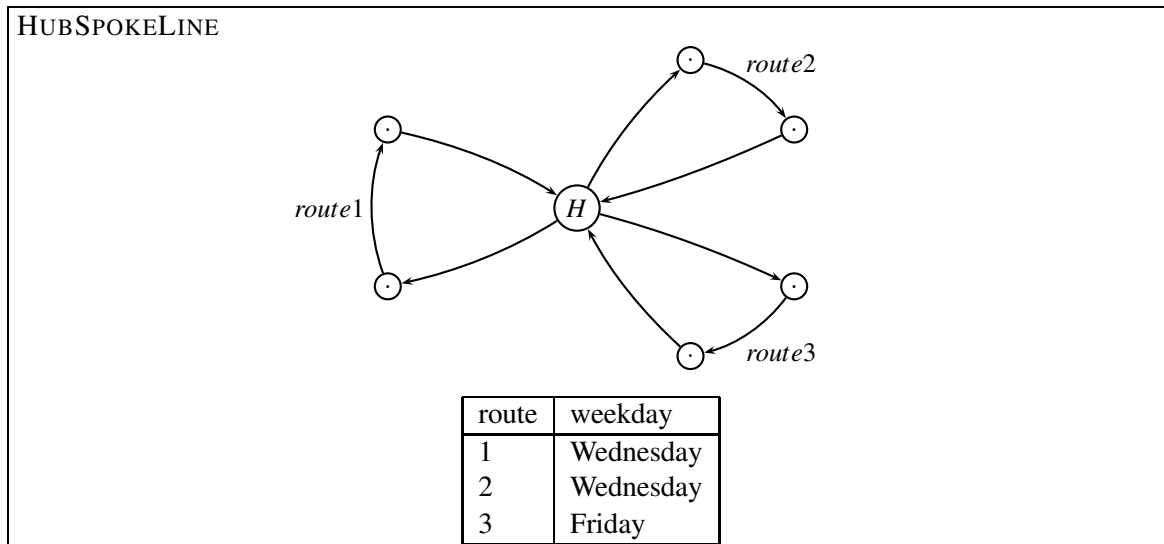
We solve the shortest path problem using Dijkstra:

considered node	distance				
	A	B	C	D	E
init	∞	0	∞	∞	∞
B			6	2	
D					3
E		10			
C					
A					

The shortest path is $BDEA$ with distance $z = 10$. The reduced cost is now $\bar{c}_3 = z \cdot w_3 - \bar{y}_3 = 10 \cdot 4 - 100 = -60$. ■

Answer 12 The path $BDEA$ for commodity 3 should be added, since it has most negative reduced cost. ■

Modeling a hub port

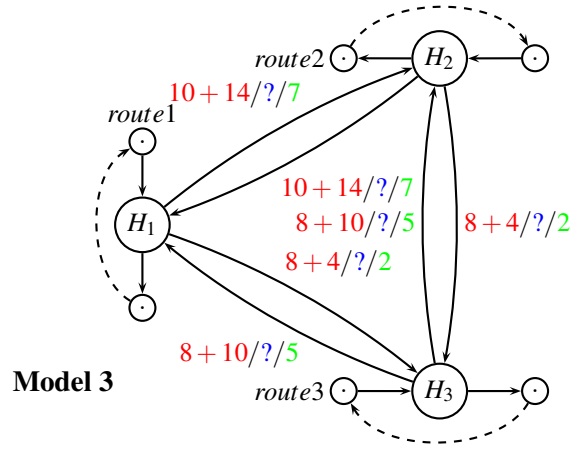


Answer 13 We consider the three models separately.

Model 3

transshipment cost	transshipment time	crane capacity	storage capacity	storage cost
yes	yes	no	no	yes

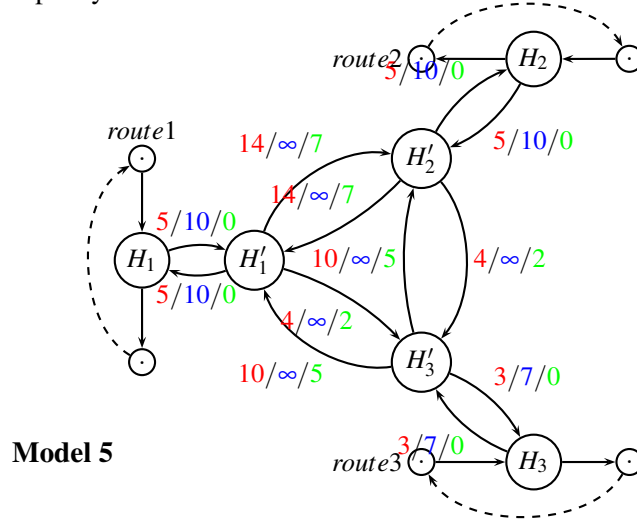
In this formulation it is possible to handle transshipment cost and time on edges between H_i and H_j . The time is set to the number of days between arrival of route i and route j . The cost is set to the unloading cost for route i plus the loading cost for route j . To the cost we add the storage cost for the given number of days. However we cannot handle crane capacity since e.g. edge (H_1, H_2) and edge (H_1, H_3) share the capacity of the crane. The same argument can be used to show that we cannot handle storage capacity.



Model 5

transshipment cost	transshipment time	crane capacity	storage capacity	storage cost
yes	yes	yes	no	yes

In this formulation it is possible to handle transshipment cost and time, as well as crane capacity. Transshipment costs and crane capacity are assigned to edges between H_i and H'_i . The time (and storage cost) is assigned to edges between H'_i and H'_j as the number of days between arrival of route i and route j . However we cannot handle storage capacity since e.g. edge (H'_1, H'_2) and edge (H'_1, H'_3) share the same storage capacity.

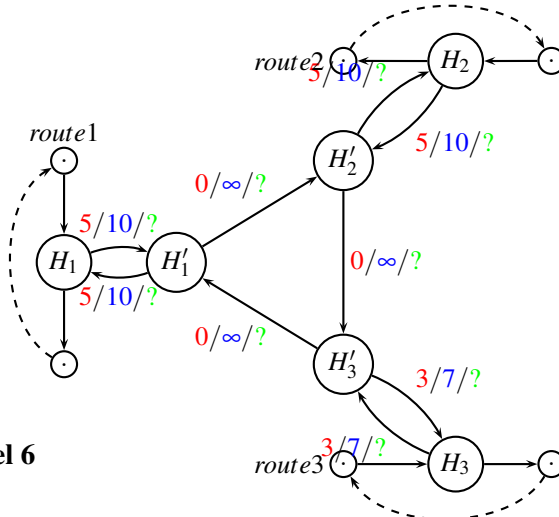


Model 5

Model 6

transshipment cost	transshipment time	crane capacity	storage capacity	storage cost
yes	no	yes	no	no

We assume that the vessels are ordered according to increasing day of arrival. In this formulation it is possible to handle transshipment cost, crane capacity. Transshipment costs and crane capacity are assigned to edges between H_i and H'_i . Storage capacity and storage cost cannot be handled, since they need to be 0 from H'_1 to H'_2 as well as from H'_2 to H'_1 . Transshipment times cannot be modeled, since the time between H'_1 and H'_2 is 7 days, so a transshipment time from H'_3 to H'_2 (taking 5 days) cannot be modeled.



Model 6